Национальный исследовательский университет «Высшая школа экономики»
Московский институт электроники и математики им. А. Н. Тихонова

ДОКЛАД

Классы и ООП

Java и C++ на примере Analitik Graph

Как был устроен код до перестройки и как сейчас; наследование, абстракция, полиморфизм, инкапсуляция, final и static; виды классов в Java; чем классы C++ отличаются от Java

Автор: Арсений, группа БИВ255
Проект: Analitik Graph — Java 11, Spring Boot 2.7, 74 класса в 11 пакетах
Примеры: весь код из проекта взят с номерами строк; учебные программы скомпилированы, их вывод настоящий
Москва, 2026

Содержание

1 Как было и как стало

процедурный код → объектная модель; до и после на примерах

2 Класс и объект

поля, методы, конструктор, this, ссылки, куча

3 Инкапсуляция

private, package-private, protected, public

4 Наследование

extends, super, @Override, Object, композиция

5 Абстракция

абстрактные классы и интерфейсы, как выбрать

6 Полиморфизм

динамическое связывание, перегрузка, дженерики

7 Слово final

класс, метод, поле, константа, лямбды

8 Слово static

члены класса, утилиты, фабрики

9 Виды классов и структур

enum, лямбды, вложенные, исключения, record, sealed, коллекции, паттерны

10 C++ и Java

деструкторы, virtual, срезка, = 0, множественное наследование, таблица

11 Плюсы и цена

что дала перестройка и её минусы

— Шпаргалка

всё главное на одной странице

Как читать этот доклад

Каждый раздел устроен одинаково: сначала идея простыми словами, потом учебный пример, который можно запустить, потом тот же приём в коде проекта и, наконец, два блока для запоминания. Синий блок «Запомни» — четыре фразы, которые нужно знать наизусть. Жёлтый блок «Если спросят» — готовый ответ на 20 секунд, его удобно проговаривать вслух.

Код в рамке с красной подписью «Было» — из старой версии проекта, с зелёной «Стало» — из текущей, с настоящими номерами строк. Чёрный блок под учебной программой — её настоящий вывод. Красные плашки «javac» и «g++» — реальные сообщения компиляторов, когда правило нарушено: их полезно помнить, они сразу показывают, что именно запрещает каждое слово.

Последняя страница — шпаргалка. Её стоит перечитать последней перед ответом.

1 Как было и как стало

Главная мысль раздела: раньше классы в проекте были, но работали как «папки» для функций. Теперь каждый класс — это отдельное понятие со своими данными, поведением и местом в иерархии.

1.1. Что было: 18 больших классов-«папок»

До перестройки серверная часть «мозга» состояла из 18 файлов и 6 732 строк. Формально это были классы, потому что в Java по-другому нельзя: любой код обязан лежать внутри класса. Но по сути это был **процедурный стиль**. Класс `Stats` на 652 строки был просто набором статических функций: `Stats.spearman(...)`, `Stats.welch(...)`, `Stats.chiSquare(...)`. Объект `Stats` никто никогда не создавал — у него даже конструктор был закрыт. То же самое с `Query` (926 строк), `Analytics` (842 строки) и `BrainService` (1 131 строка).

Результат теста хранился во вложенном классе `Stats.Test`, у которого были только поля и ни одного метода. Такой класс называют **классом-структурой** или «мешком с данными»: он хранит значения, но ничего не умеет делать сам.

Было · класс-структура: только поля, поведения нет · Stats.java, строки 29–43

```
29  public static class Test {
30      public String test;                 // название теста
31      public double stat;                 // значение статистики (t, χ², F …)
32      public double df1 = Double.NaN;     // степени свободы
33      public double df2 = Double.NaN;
34      public double p = 1;                // p-значение
35      public double q = 1;                // p после поправки Бенджамини–Хохберга
36      public String effectName;           // ρ, V, η, g или h
37      public double effect;               // размер эффекта
38      public double lo = Double.NaN;      // 95% доверительный интервал эффекта
39      public double hi = Double.NaN;
40      public int n;
41      public String level = "none";       // strong / medium / weak / none
42      public double strength;             // 0..1, для толщины линии на графе
43  }
```

Сам расчёт жил отдельно — в статической функции, которая создавала пустой `Test` и заполняла его поля снаружи, как бланк:

Было · функция заполняет чужой объект · Stats.java, строки 269–302

```java
public static Test spearman(double[] a, double[] b) {
    List<double[]> pairs = new ArrayList<>();
    for (int i = 0; i < a.length; i++) {
        if (!Double.isNaN(a[i]) && !Double.isNaN(b[i])) pairs.add(new double[]{a[i], b[i]});
    }
    int n = pairs.size();
    double[] x = new double[n];
    double[] y = new double[n];
    for (int i = 0; i < n; i++) {
        x[i] = pairs.get(i)[0];
        y[i] = pairs.get(i)[1];
    }

    Test t = new Test();
    t.test = "Корреляция Спирмена";
    t.effectName = "ρ";
    t.n = n;
    if (n < 5) return t;

    double rho = pearson(ranks(x), ranks(y));
    rho = Math.max(-0.999999, Math.min(0.999999, rho));   // чтобы не делить на ноль ниже
    t.effect = rho;
    t.stat = rho * Math.sqrt((n - 2) / (1 - rho * rho));
    t.df1 = n - 2;
    t.p = tP(t.stat, n - 2);

    double z = 0.5 * Math.log((1 + rho) / (1 - rho));
    double se = Math.sqrt(1.06 / Math.max(1, n - 3));
    t.lo = Math.tanh(z - 1.96 * se);
    t.hi = Math.tanh(z + 1.96 * se);
    t.strength = Math.abs(rho);
    t.level = levelCorr(Math.abs(rho));
    return t;
}
```

Шкала силы связи («слабая / средняя / сильная») была записана двумя почти одинаковыми функциями — для корреляций и для разниц. Отличались только числа-пороги:

Было · дублирование · Stats.java, строки 629–644

```java
public static String levelCorr(double r) {
    r = Math.abs(r);
    if (r >= 0.5) return "strong";
    if (r >= 0.3) return "medium";
    if (r >= 0.1) return "weak";
    return "none";
}

/** Для g и h. */
public static String levelD(double d) {
    d = Math.abs(d);
    if (d >= 0.8) return "strong";
    if (d >= 0.5) return "medium";
    if (d >= 0.2) return "weak";
    return "none";
}
```

1.2. Три главные проблемы старого кода

Проблема 1. Проверки типа через instanceof

Когда нужно было подписать лист графа, код спрашивал у объекта «а ты случайно не тест долей?» и вручную приводил тип. Каждое новое различие между тестами добавляло ещё один `if (t instanceof ...)`. Если бы появился третий тест сравнения, пришлось бы искать и править все такие места.

Было · проверка типа и приведение · BrainService.java, строки 834–836

```
834  if (t instanceof Stats.Prop) {
835      Stats.Prop pr = (Stats.Prop) t;
836      n.sub = purchaseVerb(m.name) + " " + pct(pr.pA) + (hasRest ? " · " + sgnPP(pr.pA - pr.pB) :
     "") + stars;
```

Было · ещё две проверки того же типа ниже · BrainService.java, строки 859–863

```
859  if (hasRest) {
860      String statName = t instanceof Stats.Prop ? "z = " : "t = ";
861      String df = Double.isNaN(t.df1) ? "" : ", df = " + fmt(t.df1);
862      boolean showCi = !Double.isNaN(t.lo) && !(t instanceof Stats.Prop);
863      String ci = showCi ? "  [" + fmt(t.lo) + "; " + fmt(t.hi) + "]" : "";
```

Проблема 2. Длинный switch по строкам

ИИ-инструменты выбирались огромным `switch` на 17 веток. Чтобы добавить инструмент, нужно было дописать ветку в середину метода, а ошибку в названии компилятор не видел.

Было · выбор инструмента · AiTools.java, строки 37–50

```
37  /** Выполнить инструмент name с аргументами args. Возвращает JSON-строку для модели. */
38  String run(Dataset ds, String name, JsonNode args, BiConsumer<String, Object> emit) throws
    Exception {
39      if (ds == null) throw new IllegalStateException("Нет загруженных данных");
40      switch (name) {
41          case "pivot_table":
42              return show(Query.pivot(ds, JSON.treeToValue(args, Query.PivotSpec.class)), emit);
43          case "chart":
44              return show(Query.chart(ds, JSON.treeToValue(args, Query.ChartSpec.class)), emit);
45          case "compare_segments":
46              return show(Query.compare(ds, args.path("metric").asText(),
    args.path("group_by").asText(), conds(args)), emit);
47          case "funnel":
48              return show(Query.funnel(ds, args.path("by").asText(null), conds(args)), emit);
49          case "data_quality":
50              return show(Analytics.quality(ds), emit);
```

Проблема 3. Формула перечитывалась для каждой строки

Старый класс `Formula` хранил текст формулы и позицию чтения. Метод `eval(i)` для каждой строки таблицы начинал разбор текста заново с позиции 0. Для тысячи строк формула разбиралась тысячу раз.

Было · разбор заново на каждой строке · Formula.java, строки 21–43

```
21  class Formula {
22
23      private final String src;
24      private final Dataset ds;
25      private int pos;   // где сейчас читаем формулу
26      private int row;   // для какой строки считаем
27
28      Formula(String src, Dataset ds) {
29          this.src = src == null ? "" : src;
30          this.ds = ds;
31      }
32
33      /** Значение формулы для строки i. */
34      Object eval(int i) {
35          row = i;
36          pos = 0;
37          Object value = or();
38          skipSpaces();
39          if (pos < src.length()) {
40              throw new IllegalArgumentException("Ошибка в формуле около: «" + src.substring(pos)
    + "»");
41          }
42          return value;
43      }
```

1.3. Что стало: 74 класса в 11 пакетах

После перестройки код разложен по смыслу на 11 пакетов, а тип каждого класса виден прямо в имени файла. Всего 74 класса и 7 199 строк. Строк стало чуть больше, потому что у каждого класса теперь своё объявление, конструктор и импорты, — зато каждый файл короткий и отвечает за одну вещь.

18 → 74
классов

1 → 11
пакетов

0 → 6
абстрактных классов

0 → 4
интерфейсов

Приставка	Сколько	Что это значит
`Final…`	51	конечный класс: объявлен с `final`, от него нельзя наследоваться
`Abstract…`	6	абстрактный родитель: общие поля и методы, объект создать нельзя
`Interface…`	4	интерфейс: список методов, которые класс обещает реализовать
`Enum…`	4	перечисление: закрытый набор значений со своими полями и методами
`Service…`	5	сервис Spring: создаётся один раз и передаётся другим через конструктор
`Controller…`	2	контроллер: принимает HTTP-запросы от браузера
`Exception…`	2	собственные исключения: ошибки данных (400) и «набор не найден» (404)

Честно о приставках

В обычных Java-проектах приставки `Final` или `Abstract` в имени класса не пишут: тип и так виден в объявлении. Мы добавили их намеренно, для учебной наглядности — чтобы по списку файлов сразу было видно, где наследование и где конечные классы. Если спросят, так и отвечай: «это соглашение проекта для защиты, в промышленном коде было бы `WelchTest`, а не `FinalWelchTest`».

1.4. Куда что переехало

Было	Стало	Что изменилось по сути
`Stats` (652 строки)	пакет `mathematics`: 14 классов	каждый тест стал своим классом; общий родитель, интерфейс, перечисление шкалы силы
`BrainService` (1 131)	`ServiceBrainGraph` + 4 класса в `service`	сервис только координирует, построение веток, связей и находок — в отдельных классах
`Query` (926)	`FinalQueryEngine` + 4 класса-описания	вложенные классы-описания вынесены в свои файлы
`Analytics` (842)	пакет `analysis`: 9 классов	шесть анализов — наследники `AbstractAnalitikAnalysis` с шаблонным методом
`Formula` (331)	пакет `formula`: 10 классов	текст разбирается один раз в дерево объектов; операторы и функции — перечисления
`DataLoader` (634)	пакет `data`: 9 классов	CSV и JSON — две реализации интерфейса `InterfaceDataParser`
`Graph` (161)	пакет `graph`: 9 классов	вложенные классы узла, связи, фильтра стали отдельными файлами
`AiTools` и др.	пакет `ai`: 6 классов	`switch` заменён словарём лямбд `Map<String, InterfaceAiTool>`
`BrainController`, `AiController`	пакет `api`: 4 класса	общий абстрактный родитель с обработкой ошибок
—	пакет `exception`: 2 класса	свои типы ошибок вместо общих `IllegalArgumentException`

1.5. Одна задача — до и после

Тест Спирмена

Было: функция создаёт пустой бланк и заполняет его. Стало: тест — это класс. Конструктор сам считает статистику, а уровень силы определяет унаследованный метод `classifyEffect()`, который берёт шкалу из перечисления. Функции `levelCorr` и `levelD` больше не нужны.

Стало · тест — это объект, который сам себя считает · mathematics/FinalSpearmanTest.java, строки 3–26

```
3  public final class FinalSpearmanTest extends AbstractCorrelationTest {
4
5      public FinalSpearmanTest(double[] a, double[] b) {
6          super("Корреляция Спирмена", "ρ");
7          double[][] pairs = FinalAnalitikMathematics.bothPresent(a, b);
8          double[] x = pairs[0];
9          double[] y = pairs[1];
10         n = x.length;
11         if (n < 5) return;
12
13         double rho = FinalAnalitikMathematics.pearson(FinalAnalitikMathematics.ranks(x),
   FinalAnalitikMathematics.ranks(y));
14         rho = Math.max(-0.999999, Math.min(0.999999, rho));
15         effect = rho;
16         stat = rho * Math.sqrt((n - 2) / (1 - rho * rho));
17         df1 = n - 2;
18         p = FinalProbabilityDistributions.tP(stat, n - 2);
19
20         double z = 0.5 * Math.log((1 + rho) / (1 - rho));
21         double se = Math.sqrt(1.06 / Math.max(1, n - 3));
22         lo = Math.tanh(z - 1.96 * se);
23         hi = Math.tanh(z + 1.96 * se);
24         classifyEffect();
25     }
26 }
```

Подпись листа графа

Было: `if (t instanceof Stats.Prop)` и приведение типа. Стало: переменная имеет общий тип, и каждый тест сам знает, как себя подписать. Это и есть полиморфизм (подробно — в разделе 6).

Стало · ни одного instanceof · service/FinalBranchBuilder.java, строки 554–555

```
554 n.sub = t.leafSubtitle(m.name, hasRest) + stars;
555 n.stats.addAll(t.leafStats(m.name, mt.segment, mt.others, hasRest));
```

ИИ-инструменты

Стало · вместо switch — поиск в словаре · ai/ServiceAiTools.java, строки 46–51

```
46 String run(FinalDataset ds, String name, JsonNode args, BiConsumer<String, Object> emit) throws
   Exception {
47     if (ds == null) throw new IllegalStateException("Нет загруженных данных");
48     InterfaceAiTool tool = tools.get(name);
49     if (tool == null) throw new ExceptionAnalitikData("Неизвестный инструмент " + name);
50     return tool.run(ds, args, emit);
51 }
```

Формула

Стало · дерево строится один раз и вычисляется для каждой строки · query/FinalQueryEngine.java, строки 739–750

```
739 InterfaceFormulaNode f = FinalFormulaParser.parse(formula, ds);
740
741 List<String> values = new ArrayList<>();
742 for (int i = 0; i < ds.rows; i++) {
743     Object v;
744     try {
745         v = f.evaluate(i);
746     } catch (ArithmeticException e) {
747         v = null;
748     }
749     values.add(cellText(v));
750 }
```

1.6. Почему поведение не изменилось

Перестройка кода без изменения поведения называется **рефакторингом**. Чтобы доказать, что ничего не сломано, старая и новая версии сервера запускались рядом, и в обе отправлялись одни и те же 723 запроса: загрузка данных, обзор, раскрытие веток, сводные, графики, все анализы и чат с тестовой моделью. Ответы совпали побайтно, кроме номера порта в ссылках. Отдельно проверено 80 формул — результаты одинаковые. Этот приём называется **golden master** («эталонный вывод»): старая версия служит эталоном для новой.

ЗАПОМНИ

1. Раньше классы были «папками» со static-функциями — это процедурный стиль в синтаксисе Java.
2. Три болезни старого кода: `instanceof`, длинный `switch`, повторный разбор формулы.
3. Теперь 74 класса в 11 пакетах; тип класса — первое слово имени (приставка — наше учебное соглашение).
4. Поведение доказано одинаковым: 723 запроса и 80 формул дают тот же ответ (golden master).

ЕСЛИ СПРОСЯТ — ОТВЕТ ЗА 20 СЕКУНД

«Раньше код был процедурным: большие классы со статическими функциями и классы-структуры без поведения. Я перестроил его в объектную модель: каждый тест, анализ и узел формулы стал своим классом, общие части ушли в абстрактных родителей, а instanceof и switch заменил полиморфизмом. Поведение проверено на 723 запросах».

2 Класс и объект

Класс — это чертёж. Объект — это конкретная вещь, построенная по чертежу. Чертёж «Колонка» один, а колонок «Выручка», «NPS», «Город» — сколько угодно.

В классе описываются **поля** — данные, которые хранит каждый объект, и **методы** — действия, которые объект умеет выполнять со своими данными. Особый метод без возвращаемого типа и с именем класса называется **конструктором**: он вызывается один раз, в момент создания объекта оператором `new`, и приводит объект в правильное начальное состояние.

Слово `this` внутри класса означает «этот самый объект». Запись `this.name = name` читается так: «в поле name этого объекта положи значение параметра name». Запись `this(name, new double[0])` в первой строке конструктора вызывает другой конструктор того же класса — так не приходится дублировать код.

Класс Column: поля, два конструктора, методы · запускается как есть

```
class Column {
    private final String name;
    private double[] values;

    Column(String name) {
        this(name, new double[0]);
    }

    Column(String name, double[] values) {
        this.name = name;
        this.values = values;
    }

    double mean() {
        double sum = 0;
        for (double v : values) sum += v;
        return values.length == 0 ? Double.NaN : sum / values.length;
    }

    void add(double v) {
        values = java.util.Arrays.copyOf(values, values.length + 1);
        values[values.length - 1] = v;
    }

    String name() {
        return name;
    }
}

public class Main {
    public static void main(String[] args) {
        Column revenue = new Column("Выручка", new double[]{100, 200, 300});
        Column same = revenue;
        same.add(400);
        System.out.println(revenue.name() + ": среднее " + revenue.mean());
        System.out.println("Один и тот же объект? " + (revenue == same));

        Column empty = new Column("Пустая");
        System.out.println(empty.name() + ": среднее " + empty.mean());
    }
}
```

Вывод программы

```
Выручка: среднее 250.0
Один и тот же объект? true
Пустая: среднее NaN
```

2.1. Что происходит в памяти

В Java все объекты живут в **куче** (heap) — общей области памяти. Переменная `revenue` хранит не сам объект, а **ссылку** на него, то есть «адрес». Строка `Column same = revenue;` не копирует колонку, а копирует ссылку: теперь две переменные указывают на один объект. Поэтому `same.add(400)` изменило и `revenue`, а сравнение `revenue == same` дало `true`: оператор `==` для объектов сравнивает ссылки, а не содержимое. Для сравнения содержимого служит метод `equals`.

Удалять объекты вручную не нужно и нельзя. Когда на объект не остаётся ни одной ссылки, его находит и освобождает **сборщик мусора** (garbage collector). Это главное отличие от C++, где объект удаляет программист (раздел 10).

Примитивные типы — `int`, `double`, `boolean`, `char`, `long`, `byte`, `short`, `float` — не объекты. Они хранятся прямо в переменной и копируются по значению. У каждого есть класс-обёртка (`Integer`, `Double` …), который нужен, например, в коллекциях: `List<Integer>`, но не `List<int>`.

2.2. Класс в нашем проекте

Настоящая колонка данных — класс `FinalColumn`. Поле `name` помечено `final`: имя задаётся в конструкторе и больше не меняется. Методы `analyzable()` и `isDim()` отвечают на вопросы о колонке, спрашивая перечисление `EnumColumnType`.

Стало · data/FinalColumn.java — файл целиком

```
1  package com.analytics.brain.data;
2
3  import java.util.List;
4
5  public final class FinalColumn {
6
7      public final String name;
8      public EnumColumnType type;
9      public double[] num;
10     public String[] cat;
11     public List<String> levels;
12     public int missing;
13     public int unique;
14     public String granularity;
15     public boolean countLike;
16
17     public FinalColumn(String name) {
18         this.name = name;
19     }
20
21     public boolean analyzable() {
22         return type != null && type.analyzable();
23     }
24
25     public boolean isDim() {
26         return type != null && type.dimension();
27     }
28
29     public String typeRu() {
30         return type == null ? EnumColumnType.TEXT.russianName() : type.russianName();
31     }
32 }
```

ЗАПОМНИ

1. Класс — чертёж, объект — экземпляр; объект создаётся через `new` и конструктор.
2. Переменная хранит ссылку; `b = a` — две ссылки на один объект; `==` сравнивает ссылки.
3. Объекты живут в куче, освобождает их сборщик мусора; деструкторов в Java нет.
4. `this` — текущий объект; `this(...)` — вызов соседнего конструктора.

3 Инкапсуляция и модификаторы доступа

Инкапсуляция — это правило «данные и действия с ними лежат вместе, а снаружи виден только необходимый минимум». Инструмент инкапсуляции — модификаторы доступа.

Модификатор	Сам класс	Тот же пакет	Наследник в другом пакете	Все остальные
`private`	да	нет	нет	нет
без слова (package-private)	да	да	нет	нет
`protected`	да	да	да	нет
`public`	да	да	да	да

Запомнить таблицу проще всего как лестницу: каждая следующая ступень открывает доступ ещё одной группе. `private` — только я; без модификатора — я и соседи по пакету; `protected` — плюс мои наследники; `public` — все.

В проекте инкапсуляция видна в нескольких местах. Конструктор парсера формул закрыт: создать объект `new FinalFormulaParser(...)` снаружи нельзя, можно только вызвать статический метод `parse`, который сам создаст парсер, разберёт текст и вернёт готовое дерево. Так внешний код не может оставить парсер в полуготовом состоянии.

Стало · закрытый конструктор и фабричный метод · formula/FinalFormulaParser.java, строки 24–37

```
24  private FinalFormulaParser(String src, FinalDataset ds) {
25      this.src = src == null ? "" : src;
26      this.ds = ds;
27  }
28
29  public static InterfaceFormulaNode parse(String src, FinalDataset ds) {
30      FinalFormulaParser parser = new FinalFormulaParser(src, ds);
31      InterfaceFormulaNode root = parser.or();
32      parser.skipSpaces();
33      if (parser.pos < parser.src.length()) {
34          throw new ExceptionAnalitikData("Ошибка в формуле около: «" +
parser.src.substring(parser.pos) + "»");
35      }
36      return root;
37  }
```

```
javac:  Parser() has private access in Parser
new Parser();
```

Поле `ds` в `AbstractAnalitikAnalysis` объявлено `protected final`: наследники-анализы читают набор данных, но никто не может подменить его после создания.

Отдельно стоит сказать про публичные поля в классах графа и тестов — например, `public double p`. Это осознанный компромисс: библиотека Jackson превращает объект в JSON для браузера, читая именно публичные поля. В учебниках обычно советуют закрыть поле и дать метод-геттер `getP()`; у нас же такие классы — это «объекты передачи данных» (DTO), и геттеры добавили бы только лишний код.

ЗАПОМНИ

1. Инкапсуляция: данные и методы вместе, наружу — минимум.
2. Лестница доступа: `private` → пакет → `protected` → `public`.
3. Закрытый конструктор + статический метод = контроль над созданием объекта (`FinalFormulaParser.parse`).
4. Публичные поля в DTO — ради Jackson; в «обычных» классах поля закрывают.

ЕСЛИ СПРОСЯТ — ОТВЕТ ЗА 20 СЕКУНД

«Инкапсуляция — это сокрытие внутреннего устройства. В Java её дают модификаторы private, package-private, protected и public. Например, конструктор FinalFormulaParser закрыт, и создать парсер можно только через метод parse, поэтому объект никогда не бывает недособранным».

4 Наследование

Наследование — это когда новый класс получает поля и методы готового класса и добавляет своё. Проверка правильности: между ними должно быть отношение «является» (is-a). Тест Уэлча является тестом.

4.1. Синтаксис: extends и super

Наследник объявляется словом `extends`: `class FinalWelchTest extends AbstractSegmentComparisonTest`. Класс-родитель ещё называют **суперклассом** или базовым, наследника — **подклассом** или производным.

Конструкторы не наследуются. Зато конструктор наследника обязан первым делом вызвать конструктор родителя через `super(...)`. Если этого не написать, Java сама вставит вызов `super()` без аргументов — и если у родителя такого конструктора нет, будет ошибка компиляции. Поэтому объект всегда собирается **сверху вниз**: сначала часть самого дальнего предка, потом следующего, и последней — часть самого класса.

Порядок вызова конструкторов в цепочке из трёх классов · запускается как есть

```
abstract class AbstractTest {
    protected final String name;

    protected AbstractTest(String name) {
        this.name = name;
        System.out.println("1. AbstractTest: имя = " + name);
    }
}

abstract class AbstractSegmentTest extends AbstractTest {
    protected final int nA;
    protected final int nB;

    protected AbstractSegmentTest(String name, int nA, int nB) {
        super(name);
        this.nA = nA;
        this.nB = nB;
        System.out.println("2. AbstractSegmentTest: n = " + (nA + nB));
    }
}

final class WelchTest extends AbstractSegmentTest {
    final double t;

    WelchTest(double[] a, double[] b) {
        super("t-тест Уэлча", a.length, b.length);
        this.t = 2.41;
        System.out.println("3. WelchTest: t = " + t);
    }
}

public class Main {
    public static void main(String[] args) {
        new WelchTest(new double[]{1, 2, 3}, new double[]{4, 5});
    }
}
```

Вывод программы

```
1. AbstractTest: имя = t-тест Уэлча
2. AbstractSegmentTest: n = 5
3. WelchTest: t = 2.41
```

Именно так устроены наши тесты. `FinalWelchTest` передаёт наверх название теста, символ эффекта и размеры групп, а промежуточный родитель передаёт название и символ ещё выше:

Стало · промежуточный родитель · mathematics/AbstractSegmentComparisonTest.java, строки 10–15

```
10  protected AbstractSegmentComparisonTest(String test, String effectName, int nA, int nB) {
11      super(test, effectName);
12      this.nA = nA;
13      this.nB = nB;
14      this.n = nA + nB;
15  }
```

Стало · первая строка — super(...) · mathematics/FinalWelchTest.java, строки 17–22

```
17  public FinalWelchTest(double[] a, double[] b) {
18      super("t-тест Уэлча", "g", a.length, b.length);
19      meanA = FinalAnalitikMathematics.mean(a);
20      meanB = FinalAnalitikMathematics.mean(b);
21      boolean noBase = b.length == 0 || Math.abs(meanB) < 1e-12;
22      lift = noBase ? Double.NaN : (meanA - meanB) / Math.abs(meanB);
```

4.2. Что получает наследник

Наследник получает все поля и методы родителя. Поля `private` в объекте есть, но обращаться к ним напрямую наследник не может — только через методы родителя. `protected`-члены видны наследнику напрямую. Статические методы не переопределяются, а «скрываются»: вызов зависит от типа переменной, а не объекта.

Наследник может **переопределить** (override) метод родителя — написать свою версию с той же сигнатурой. Над такой версией ставят аннотацию `@Override`. Она не обязательна, но очень полезна: если ошибиться в имени или параметрах, компилятор сразу скажет, что метод ничего не переопределяет.

```
javac:  method does not override or implement a method from a supertype
@Override public String toStrng() { return "t"; }
```

Правила переопределения: имя и параметры те же; возвращаемый тип тот же или более узкий (например, наследник вместо `Object` может вернуть `String`); доступ нельзя сузить — `public` метод нельзя сделать `protected`; метод с `final` переопределить нельзя вообще.

```
javac:  symbol() in WelchTest cannot override symbol() in AbstractTest — attempting to assign weaker
access privileges; was public
protected String symbol() { return "t"; }
```

Из переопределённого метода можно вызвать версию родителя: `super.describe()`. Это делает `TwoProportionTest` в примере из раздела 6.

4.3. Одиночное наследование и класс Object

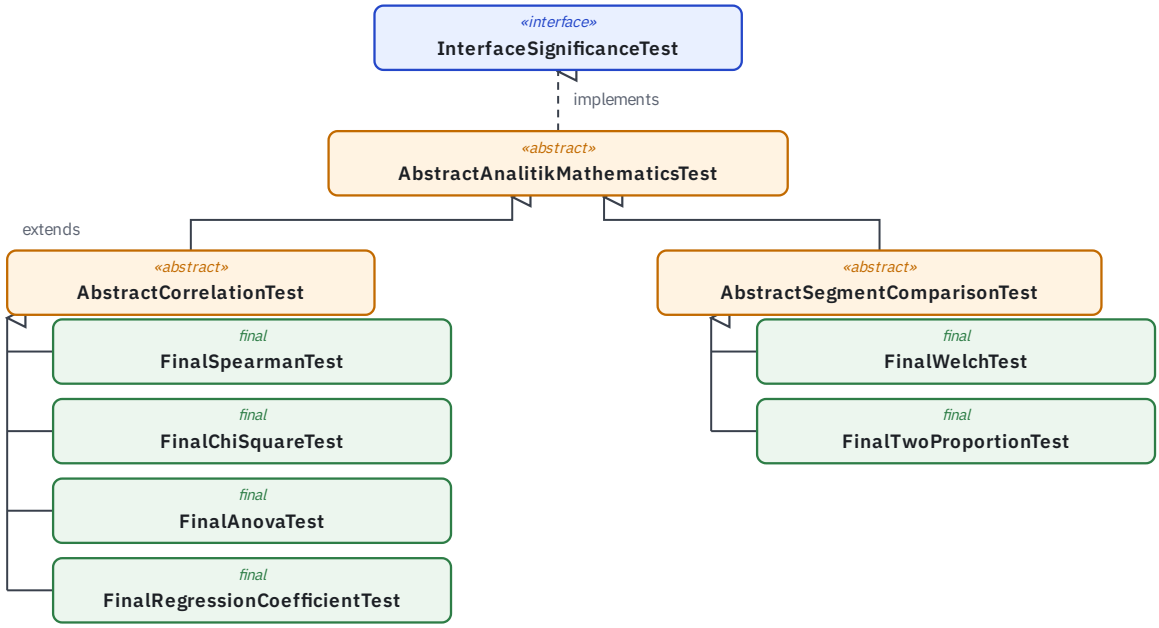
В Java у класса может быть **только один** родитель. Запись `class C extends A, B` — синтаксическая ошибка. Так язык избегает «ромба»: если бы A и B оба имели метод `run()`, было бы неясно, чей брать. Нужную гибкость дают интерфейсы: реализовать можно сколько угодно (раздел 5).

Если `extends` не написан, класс неявно наследует `java.lang.Object`. Поэтому у любого объекта есть методы `toString()`, `equals()`, `hashCode()`, `getClass()`. У `AbstractAnalitikMathematicsTest` цепочка такая: `Object` → `AbstractAnalitikMathematicsTest` → `AbstractSegmentComparisonTest` → `FinalWelchTest`.

```
javac:  '{' expected
class C extends A, B {}
```

4.4. Все цепочки наследования в проекте

Родитель	Наследники	Связь
`InterfaceSignificanceTest`	`AbstractAnalitikMathematicsTest`	implements
`AbstractAnalitikMathematicsTest`	`AbstractCorrelationTest`, `AbstractSegmentComparisonTest`	extends
`AbstractCorrelationTest`	`FinalSpearmanTest`, `FinalChiSquareTest`, `FinalAnovaTest`, `FinalRegressionCoefficientTest`	extends
`AbstractSegmentComparisonTest`	`FinalWelchTest`, `FinalTwoProportionTest`	extends
`AbstractAnalitikAnalysis`	`FinalDriversAnalysis`, `FinalAbcAnalysis`, `FinalTrendAnalysis`, `FinalFinanceAnalysis`, `FinalQualityAnalysis`, `AbstractAnomalyAnalysis`	extends
`AbstractAnomalyAnalysis`	`FinalGroupAnomalyAnalysis`, `FinalRecordAnomalyAnalysis`	extends
`AbstractApiController`	`ControllerBrainApi`, `ControllerAiApi`	extends
`InterfaceFormulaNode`	5 узлов: `FinalConstantNode`, `FinalCellNode`, `FinalUnaryNode`, `FinalBinaryNode`, `FinalFunctionNode`	implements
`InterfaceDataParser`	`FinalCsvParser`, `FinalJsonParser`	implements
`IllegalArgumentException` (JDK)	`ExceptionAnalitikData`	extends
`NoSuchElementException` (JDK)	`ExceptionDatasetNotFound`	extends



Главная иерархия: тесты значимости. Треугольная стрелка идёт от наследника к родителю, пунктир — реализация интерфейса.

4.5. Наследование или композиция

Наследование — не единственный способ переиспользовать код. **Композиция** — это когда объект содержит другой объект в поле, отношение «имеет» (has-a). Узел формулы `FinalBinaryNode` не наследует оператор, а *имеет* его в поле и два дочерних узла. Правило из книги «Effective Java»: предпочитайте композицию

наследованию, если нет настоящего отношения «является». У нас наследование используется только там, где оно честное: тест Уэлча — это тест, анализ ABC — это анализ.

Стало · композиция: объект содержит другие объекты · formula/FinalBinaryNode.java — файл целиком

```java
package com.analytics.brain.formula;

public final class FinalBinaryNode implements InterfaceFormulaNode {

    private final EnumFormulaOperator operator;
    private final InterfaceFormulaNode left;
    private final InterfaceFormulaNode right;

    public FinalBinaryNode(EnumFormulaOperator operator, InterfaceFormulaNode left,
InterfaceFormulaNode right) {
        this.operator = operator;
        this.left = left;
        this.right = right;
    }

    @Override
    public Object evaluate(int row) {
        Object a = left.evaluate(row);
        Object b = right.evaluate(row);
        return operator.apply(a, b);
    }
}
```

ЗАПОМНИ

1. `extends` — наследование класса, только один родитель; корень всех классов — `Object`.
2. Конструкторы не наследуются; первая строка конструктора наследника — `super(...)`; сборка идёт сверху вниз.
3. `@Override` — проверка компилятором; доступ сузить нельзя, `final`-метод переопределить нельзя.
4. Наследование = «является», композиция = «имеет»; `FinalBinaryNode` — пример композиции.

ЕСЛИ СПРОСЯТ — ОТВЕТ ЗА 20 СЕКУНД

«Наследование — это получение полей и методов родителя через extends. В Java один родитель у класса, но много интерфейсов. У нас FinalWelchTest наследует AbstractSegmentComparisonTest, тот — AbstractAnalitikMathematicsTest; конструктор наследника первой строкой вызывает super(...)».

5 Абстракция: абстрактные классы и интерфейсы

Абстракция — это умение говорить о вещи по её главным свойствам, не вдаваясь в детали. «Статистический тест» — абстракция: у любого теста есть p-значение, но формула у каждого своя.

5.1. Абстрактный класс

Класс с модификатором `abstract` — это **недостроенный** класс. Создать его объект нельзя, он существует только чтобы от него наследовались. В нём может быть всё, что в обычном классе: поля, конструкторы, готовые методы. И ещё — **абстрактные методы**: у них есть объявление, но нет тела, вместо фигурных скобок стоит точка с запятой. Абстрактный метод — это обязательство: каждый не-абстрактный наследник *обязан* его написать.

Зачем конструктор классу, объект которого создать нельзя? Его вызывают наследники через `super(...)`, чтобы заполнить общие поля. У `AbstractAnalitikMathematicsTest` конструктор `protected` — снаружи его всё равно не вызвать.

Стало · общий родитель всех тестов · mathematics/AbstractAnalitikMathematicsTest.java — файл целиком

```java
package com.analytics.brain.mathematics;

public abstract class AbstractAnalitikMathematicsTest implements InterfaceSignificanceTest {

    public static final double ALPHA = 0.05;

    public String test;
    public double stat;
    public double df1 = Double.NaN;
    public double df2 = Double.NaN;
    public double p = 1;
    public double q = 1;
    public String effectName;
    public double effect;
    public double lo = Double.NaN;
    public double hi = Double.NaN;
    public int n;
    public String level = "none";
    public double strength;

    protected AbstractAnalitikMathematicsTest(String test, String effectName) {
        this.test = test;
        this.effectName = effectName;
    }

    protected abstract EnumEffectScale scale();

    protected final void classifyEffect() {
        level = scale().level(effect);
        strength = scale().strength(effect);
    }

    public final boolean significant() {
        return q < ALPHA && !"none".equals(level);
    }

    @Override
    public final double pValue() {
        return p;
    }

    @Override
    public final void acceptQValue(double q) {
        this.q = q;
    }
}
```

Разберём этот файл построчно, потому что в нём собраны почти все темы доклада. Строка 3: класс абстрактный и реализует интерфейс. Строка 5: константа `public static final`. Строки 7–19: общие поля, которые получит любой тест. Строка 21: конструктор `protected` — только для наследников. Строка 26: абстрактный метод `scale()` — какой шкалой мерить силу, решает промежуточный наследник. Строки 28 и 33: методы `final` — правило классификации и правило значимости одинаковы для всех и не могут быть изменены. Строки 37–45: реализация двух методов интерфейса, тоже `final`.

```
javac:  AbstractTest is abstract; cannot be instantiated
new AbstractTest();
```

```
javac:  WelchTest is not abstract and does not override abstract method symbol() in AbstractTest
class WelchTest extends AbstractTest {}
```

5.2. Интерфейс

Интерфейс (`interface`) — это **чистый контракт**: список методов, которые класс обещает иметь. Полей-состояний в интерфейсе нет (только константы), конструкторов нет. Все методы интерфейса по умолчанию `public abstract`, писать эти слова не нужно. Класс подключает интерфейс словом `implements` и может реализовать сколько угодно интерфейсов сразу: `class A extends B implements C, D, E`.

С Java 8 в интерфейсе можно писать методы с телом: `default`-методы (реализация по умолчанию, наследник может переопределить) и `static`-методы (вспомогательные, вызываются через имя интерфейса). С Java 9 — ещё и `private`-методы для общего кода внутри интерфейса.

Интерфейс с default- и static-методом, две реализации и лямбда · запускается как есть

```java
import java.util.ArrayList;
import java.util.List;

interface DataParser {
    List<String> parse(String text);

    default String name() {
        return getClass().getSimpleName();
    }

    static DataParser forText(String text) {
        return text.startsWith("[") ? new JsonParser() : new CsvParser();
    }
}

final class CsvParser implements DataParser {
    @Override
    public List<String> parse(String text) {
        return List.of(text.split(","));
    }
}

final class JsonParser implements DataParser {
    @Override
    public List<String> parse(String text) {
        List<String> out = new ArrayList<>();
        for (String part : text.replaceAll("[\\[\\]\"]", "").split(",")) out.add(part.trim());
        return out;
    }
}

public class Main {
    public static void main(String[] args) {
        for (String text : new String[]{"Москва,Казань,Сочи", "[\"Москва\", \"Казань\"]"}) {
            DataParser parser = DataParser.forText(text);
            System.out.println(parser.name() + " → " + parser.parse(text));
        }
        DataParser upper = text -> List.of(text.toUpperCase());
        System.out.println("лямбда → " + upper.parse("nps"));
    }
}
```

Вывод программы

```
CsvParser → [Москва, Казань, Сочи]
JsonParser → [Москва, Казань]
лямбда → [NPS]
```

В проекте четыре интерфейса, и все они очень короткие — это нормально, хороший интерфейс описывает одну способность:

Стало · mathematics/InterfaceSignificanceTest.java — файл целиком

```java
package com.analytics.brain.mathematics;

public interface InterfaceSignificanceTest {

    double pValue();

    void acceptQValue(double q);
}
```

Стало · formula/InterfaceFormulaNode.java — файл целиком

```java
package com.analytics.brain.formula;

public interface InterfaceFormulaNode {

    Object evaluate(int row);
}
```

Стало · data/InterfaceDataParser.java — файл целиком

```java
package com.analytics.brain.data;

public interface InterfaceDataParser {

    void parse(FinalDataset ds, String text) throws Exception;
}
```

Стало · ai/InterfaceAiTool.java — файл целиком

```java
package com.analytics.brain.ai;

import com.analytics.brain.data.FinalDataset;
import com.fasterxml.jackson.databind.JsonNode;

import java.util.function.BiConsumer;

@FunctionalInterface
public interface InterfaceAiTool {

    String run(FinalDataset ds, JsonNode args, BiConsumer<String, Object> emit) throws Exception;
}
```

`InterfaceSignificanceTest` нужен поправке Бенджамини–Хохберга: ей всё равно, какой это тест, лишь бы можно было взять p и отдать q. `InterfaceFormulaNode` — любой узел дерева формулы умеет вычислить себя для строки. `InterfaceDataParser` — любой парсер умеет разобрать текст в набор данных. `InterfaceAiTool` помечен `@FunctionalInterface`: в нём ровно один абстрактный метод, поэтому его можно реализовать лямбдой (раздел 9).

5.3. Абстрактный класс или интерфейс?

	Абстрактный класс	Интерфейс
Ключевое слово	`abstract class`, подключается через `extends`	`interface`, подключается через `implements`
Сколько можно у класса	один	сколько угодно
Поля	любые: обычные, `protected`, `final`	только константы `public static final`
Конструктор	есть (вызывается через `super`)	нет
Методы с телом	любые	`default`, `static`, `private` (Java 8/9+)
Модификаторы методов	любые	по умолчанию `public abstract`
Отношение	«является» (общий предок)	«умеет» (способность)
Пример у нас	`AbstractAnalitikMathematicsTest`	`InterfaceSignificanceTest`

Практическое правило: если у будущих наследников есть **общее состояние и общий код** — абстрактный класс. Если нужна только **способность**, которую могут иметь совсем разные классы, — интерфейс. Часто их сочетают, как у нас: интерфейс задаёт контракт, абстрактный класс его частично реализует, конечные классы доделывают.

ЗАПОМНИ

1. `abstract class` — объект создать нельзя; могут быть поля, конструктор, обычные и абстрактные методы.
2. Абстрактный метод без тела — обязанность для наследника; иначе наследник сам должен быть abstract.
3. `interface` — контракт; класс реализует много интерфейсов; с Java 8 есть default/static-методы.
4. Абстрактный класс — «является» и общий код; интерфейс — «умеет».

ЕСЛИ СПРОСЯТ — ОТВЕТ ЗА 20 СЕКУНД

«Абстракция — выделение главного. Абстрактный класс нельзя создать, он задаёт общий код и обязательные абстрактные методы, как AbstractAnalitikMathematicsTest с методом scale(). Интерфейс — чистый контракт, как InterfaceSignificanceTest: pValue и acceptQValue. Класс наследует один класс, но реализует много интерфейсов».

6 Полиморфизм

Полиморфизм — «много форм»: один и тот же вызов работает по-разному в зависимости от того, какой объект на самом деле лежит в переменной. Код пишется для общего типа, а работает для любого наследника.

6.1. Как это работает

Переменной родительского типа можно присвоить объект наследника: `AbstractSegmentComparisonTest t = new FinalWelchTest(a, b);` Это называется **восходящим приведением** (upcasting) и происходит автоматически. Тип переменной определяет, *какие* методы можно вызвать; реальный класс объекта определяет, *чья версия* метода выполнится.

Выбор версии происходит во время работы программы и называется **динамическим связыванием** (dynamic dispatch). В Java все обычные методы экземпляра «виртуальные» по умолчанию. Не участвуют в полиморфизме только методы `static`, `private` и `final` — их нельзя переопределить, поэтому выбирать не из чего.

Полиморфизм, вызов super и перегрузка в одном примере · запускается как есть

```java
import java.util.List;

abstract class AbstractSegmentTest {
    protected double p;

    public final boolean significant() {
        return p < 0.05;
    }

    public abstract String symbol();

    public String describe() {
        return symbol() + " · p = " + p + (significant() ? " · значимо" : " · не значимо");
    }
}

final class WelchTest extends AbstractSegmentTest {
    WelchTest(double p) {
        this.p = p;
    }

    @Override
    public String symbol() {
        return "t";
    }
}

final class TwoProportionTest extends AbstractSegmentTest {
    TwoProportionTest(double p) {
        this.p = p;
    }

    @Override
    public String symbol() {
        return "z";
    }

    @Override
    public String describe() {
        return "доли: " + super.describe();
    }
}

public class Main {
    static AbstractSegmentTest compare(boolean countLike, double p) {
        if (countLike) return new TwoProportionTest(p);
        return new WelchTest(p);
    }

    static String show(int n) {
        return "int " + n;
    }

    static String show(double x) {
        return "double " + x;
    }

    static String show(String s) {
        return "String " + s;
    }

    public static void main(String[] args) {
        List<AbstractSegmentTest> tests = List.of(compare(false, 0.01), compare(true, 0.2));
        for (AbstractSegmentTest t : tests) {
            System.out.println(t.getClass().getSimpleName() + " → " + t.describe());
        }
```

```
        System.out.println(show(5) + " | " + show(5.0) + " | " + show("5"));
    }
}
```

Вывод программы

```
WelchTest → t · p = 0.01 · значимо
TwoProportionTest → доли: z · p = 0.2 · не значимо
int 5 | double 5.0 | String 5
```

В цикле переменная `t` имеет тип `AbstractSegmentTest`, но для первого объекта вызывается `symbol()` класса `WelchTest`, а для второго — `TwoProportionTest`. Метод `describe()` написан в родителе один раз, но внутри он вызывает `symbol()` — и тоже попадает в версию наследника. `TwoProportionTest` переопределил и сам `describe()`, добавив слово «доли» и вызвав родительскую версию через `super.describe()`.

6.2. Полиморфизм в проекте

Фабричный метод compare

Метод объявлен так, что возвращает общий тип, а внутри решает, какой конкретный класс создать. Вызывающий код дальше работает с результатом, не зная, Уэлч это или тест долей.

Стало · фабричный метод · mathematics/AbstractSegmentComparisonTest.java, строки 17–20

```
public static AbstractSegmentComparisonTest compare(boolean countLike, double[] segment,
double[] others) {
    if (countLike) return FinalTwoProportionTest.ofPositive(segment, others);
    return new FinalWelchTest(segment, others);
}
```

Стало · полиморфные вызовы: leafSubtitle, leafStats, statisticSymbol · service/FinalBranchBuilder.java, строки 554–561

```
n.sub = t.leafSubtitle(m.name, hasRest) + stars;
n.stats.addAll(t.leafStats(m.name, mt.segment, mt.others, hasRest));

if (hasRest) {
    String df = Double.isNaN(t.df1) ? "" : ", df = " + fmt(t.df1);
    String ci = t.effectIntervalShown() ? "  [" + fmt(t.lo) + "; " + fmt(t.hi) + "]" : "";
    n.stat("Тест", t.test)
            .stat("Статистика", t.statisticSymbol() + fmt(t.stat) + df)
```

Выбор парсера через интерфейс

Стало · переменная типа интерфейса · data/FinalDataLoader.java, строки 113–121

```
public static void parseAuto(FinalDataset ds, String body, String contentType) throws Exception
{
    String text = body == null ? "" : body.replace("", "").trim();
    if (text.isEmpty()) throw new ExceptionAnalitikData("Источник вернул пустые данные");

    boolean json = (contentType != null && contentType.contains("json"))
            || text.startsWith("[") || text.startsWith("{");
    InterfaceDataParser parser = json ? new FinalJsonParser() : new FinalCsvParser();
    parser.parse(ds, text);
}
```

Поправка на множественные сравнения для любых тестов

Метод принимает `List<? extends InterfaceSignificanceTest>` — список чего угодно, что реализует интерфейс. Он не знает ни одного конкретного теста, но работает со всеми шестью.

Стало · код, написанный для интерфейса · mathematics/FinalMultipleComparisons.java, строки 28–33

```
28  public static void applyFdr(List<? extends InterfaceSignificanceTest> tests) {
29      double[] p = new double[tests.size()];
30      for (int i = 0; i < p.length; i++) p[i] = tests.get(i).pValue();
31      double[] q = benjaminiHochberg(p);
32      for (int i = 0; i < q.length; i++) tests.get(i).acceptQValue(q[i]);
33  }
```

Дерево формулы

Корень дерева — переменная типа `InterfaceFormulaNode`. Вызов `root.evaluate(row)` попадает в `FinalBinaryNode`, тот вызывает `evaluate` у левого и правого ребёнка, и каждый ребёнок отвечает по-своему: константа возвращает число, ячейка — значение из таблицы, функция — результат вычисления. Это классический паттерн **«Интерпретатор»**.

Стало · один из пяти узлов · formula/FinalCellNode.java — файл целиком

```
1   package com.analytics.brain.formula;
2
3   import com.analytics.brain.data.EnumColumnType;
4   import com.analytics.brain.data.FinalColumn;
5   import com.analytics.brain.data.FinalDataLoader;
6   import com.analytics.brain.data.FinalDataset;
7   import com.analytics.brain.exception.ExceptionAnalitikData;
8
9   public final class FinalCellNode implements InterfaceFormulaNode {
10
11      private final FinalDataset ds;
12      private final int index;
13      private final FinalColumn column;
14
15      public FinalCellNode(FinalDataset ds, String name) {
16          int j = ds.header.indexOf(name);
17          if (j < 0) {
18              for (int k = 0; k < ds.header.size(); k++) {
19                  if (ds.header.get(k).equalsIgnoreCase(name.trim())) j = k;
20              }
21          }
22          if (j < 0) throw new ExceptionAnalitikData("Нет колонки [" + name + "]");
23          this.ds = ds;
24          this.index = j;
25          this.column = ds.col(ds.header.get(j));
26      }
27
28      @Override
29      public Object evaluate(int row) {
30          String raw = ds.records.get(row)[index];
31          raw = raw == null ? "" : raw.trim();
32          if (column != null && column.type == EnumColumnType.NUMERIC) return column.num[row];
33
34          double d = FinalDataLoader.parseNum(raw);
35          boolean asNumber = !Double.isNaN(d) && column != null && column.type !=
    EnumColumnType.DATE;
36          return asNumber ? (Object) d : raw;
37      }
38  }
```

6.3. Перегрузка — полиморфизм времени компиляции

Перегрузка (overloading) — несколько методов с одним именем, но разными параметрами. В примере выше три метода `show`: для `int`, `double` и `String`. Какой вызвать, компилятор решает сам по типам аргументов ещё до запуска программы. Поэтому перегрузку называют **статическим** полиморфизмом, а переопределение — **динамическим**. Путать их нельзя: это самый частый вопрос на экзамене.

	Переопределение (override)	Перегрузка (overload)
Где	в наследнике	в том же классе (или наследнике)
Сигнатура	та же	другие параметры
Когда выбирается	при выполнении, по классу объекта	при компиляции, по типам аргументов
Аннотация	`@Override`	нет
Пример у нас	`leafSubtitle` в `FinalWelchTest`	два конструктора `Column` в разделе 2

6.4. Третий вид — параметрический (дженерики)

Класс или метод можно написать с **параметром типа**: `List<T>`, `Box<T>`. Один код работает с разными типами, а компилятор следит, чтобы в `List<String>` не попало число. Запись `<T extends Comparable<T>>` ограничивает T типами, которые умеют сравниваться; `? extends Number` — «любой наследник Number», такой список можно читать.

Обобщённый класс, ограниченный тип и wildcard · запускается как есть

```java
import java.util.List;

final class Box<T> {
    private final T value;

    Box(T value) {
        this.value = value;
    }

    T get() {
        return value;
    }
}

public class Main {
    static <T extends Comparable<T>> T max(List<T> items) {
        T best = items.get(0);
        for (T item : items) if (item.compareTo(best) > 0) best = item;
        return best;
    }

    static double sum(List<? extends Number> numbers) {
        double s = 0;
        for (Number n : numbers) s += n.doubleValue();
        return s;
    }

    public static void main(String[] args) {
        Box<String> name = new Box<>("NPS");
        Box<Integer> size = new Box<>(1000);
        System.out.println(name.get() + " · " + size.get());
        System.out.println(max(List.of(3, 9, 4)) + " · " + max(List.of("abc", "xyz")));
        System.out.println(sum(List.of(1, 2, 3)) + " · " + sum(List.of(0.5, 0.25)));
    }
}
```

Вывод программы

```
NPS · 1000
9 · xyz
6.0 · 0.75
```

ЗАПОМНИ

1. Полиморфизм: один вызов — поведение того класса, чей объект реально в переменной.
2. Переменная родительского типа может хранить наследника (upcasting); метод выбирается при выполнении.
3. Динамический — override; статический — overload; параметрический — generics.
4. У нас: `compare`, `parseAuto`, `applyFdr`, дерево формулы; ни одного instanceof.

ЕСЛИ СПРОСЯТ — ОТВЕТ ЗА 20 СЕКУНД

«Полиморфизм — когда код работает с общим типом, а выполняется версия конкретного класса. Например, compare возвращает AbstractSegmentComparisonTest, внутри это FinalWelchTest или FinalTwoProportionTest, и вызов leafSubtitle уходит в нужный класс. Это динамическое связывание; перегрузка — статический полиморфизм».

7 Слово final

Короткий ответ на вопрос «зачем здесь final»: final запрещает изменение. Что именно запрещено — зависит от того, где слово стоит.

Где стоит	Что запрещает	Пример в проекте	Ошибка компилятора
у класса	наследоваться от класса	`public final class FinalWelchTest`	`cannot inherit from final`
у метода	переопределять метод в наследнике	`public final boolean significant()`	`overridden method is final`
у поля	присваивать полю новое значение после конструктора	`protected final FinalDataset ds`	`cannot assign a value to final variable`
у static-поля	то же; получается константа	`public static final double ALPHA = 0.05`	`cannot assign a value to static final variable`
у локальной переменной или параметра	повторное присваивание	`final List<String> names`	`cannot assign a value to final variable`

```
javac: cannot inherit from final FinalWelchTest
class Hacked extends FinalWelchTest {}
```

```
javac: significant() in Hacked cannot override significant() in AbstractTest — overridden method is final
public boolean significant() { return false; }
```

```
javac: cannot assign a value to static final variable ALPHA
ALPHA = 0.1;
```

7.1. Зачем final у класса

Финальный класс — законченная вещь. Если бы от `FinalWelchTest` можно было наследоваться, кто-то написал бы `class Hacked extends FinalWelchTest`, переопределил бы, например, `insightText` или подменил логику, и объект «теста Уэлча» начал бы вести себя иначе. Слово `final` гарантирует: если в переменной лежит `FinalWelchTest`, то это именно тот код, что в файле. Такой же приём использует стандартная библиотека: класс `String` объявлен `final`, поэтому строки нельзя «подделать».

Второй случай — утилитные классы вроде `FinalAnalitikMathematics` или `FinalMultipleComparisons`: в них только статические методы, конструктор `private`, а класс `final`. Создать объект нельзя, наследоваться бессмысленно — всё закрыто явно.

7.2. Зачем final у метода

Метод `significant()` в `AbstractAnalitikMathematicsTest` — правило «q < 0,05 и сила не ниже слабой». Оно должно быть одинаковым для всех шести тестов, иначе граф стал бы противоречивым. `final` у метода запрещает наследнику переопределить это правило. То же с `run()` в `AbstractAnalitikAnalysis`: порядок шагов анализа фиксирован (шаблонный метод, раздел 9).

Стало · final-метод + abstract-метод · analysis/AbstractAnalitikAnalysis.java, строки 17–24

```java
public final Map<String, Object> run() {
    Map<String, Object> result = new LinkedHashMap<>();
    result.put("type", "table");
    compute(result);
    result.put("subtitle", result.get("summary"));
    result.put("keyCols", 1);
    return result;
}
```

7.3. Зачем final у поля

Поле `final` должно получить значение ровно один раз — в объявлении или в конструкторе, и потом не меняется. Это защищает от случайной перезаписи и делает класс проще: например, у узла формулы `FinalBinaryNode` все три поля `final`, дерево после построения неизменно. Поле `final` без значения в объявлении называют «пустым final» (blank final) — компилятор проверяет, что каждый конструктор его заполнил.

Ловушка: `final` у ссылки запрещает менять ссылку, но не объект. `final List<String> names` нельзя переназначить на другой список, но добавлять элементы в этот список можно. Чтобы запретить изменение содержимого, нужен неизменяемый объект, например `List.copyOf(...)`.

final-ссылка, константа, effectively final и неизменяемый список · запускается как есть

```java
import java.util.ArrayList;
import java.util.List;

public class Main {
    static final double ALPHA = 0.05;

    public static void main(String[] args) {
        final List<String> names = new ArrayList<>();
        names.add("Выручка");
        names.add("NPS");
        System.out.println("final-ссылка, но список меняется: " + names);

        int limit = 10;
        Runnable r = () -> System.out.println("лямбда видит limit = " + limit + ", ALPHA = " +
ALPHA);
        r.run();

        List<String> frozen = List.copyOf(names);
        try {
            frozen.add("Город");
        } catch (UnsupportedOperationException e) {
            System.out.println("неизменяемый список: добавить нельзя");
        }
    }
}
```

Вывод программы

```
final-ссылка, но список меняется: [Выручка, NPS]
лямбда видит limit = 10, ALPHA = 0.05
неизменяемый список: добавить нельзя
```

7.4. Effectively final и лямбды

Лямбда и анонимный класс могут читать локальные переменные метода, только если те **фактически финальны** (effectively final): слово `final` можно не писать, но значение не должно меняться после

присваивания. Переменная `limit` в примере не менялась — лямбда её видит. Стоит добавить `limit++`, и будет ошибка:

```
javac: local variables referenced from a lambda expression must be final or effectively final
int limit = 10; limit++; Runnable r = () -> System.out.println(limit);
```

Не путать

`final` — запрет изменения. `finally` — блок в `try/catch`, который выполняется всегда. `finalize()` — устаревший метод, который сборщик мусора мог вызвать перед удалением объекта; с Java 9 помечен как deprecated, использовать не нужно.

Сравнение с C++

В C++11 есть те же два смысла: `class WelchTest final` и `virtual bool significant() const final`. А роль final-поля в C++ играет `const`.

ЗАПОМНИ

1. `final` у класса — нельзя наследоваться (`FinalWelchTest`, `String`).
2. `final` у метода — нельзя переопределить (`significant()`, `run()`).
3. `final` у поля/переменной — нельзя переприсвоить; `static final` — константа (`ALPHA`).
4. final-ссылка ≠ неизменяемый объект; лямбды требуют effectively final.

ЕСЛИ СПРОСЯТ — ОТВЕТ ЗА 20 СЕКУНД

«final запрещает изменение. У класса — наследование: FinalWelchTest нельзя расширить, формулу не подменить. У метода — переопределение: significant() одинаков для всех тестов. У поля — повторное присваивание: ALPHA = 0,05 — константа. Итог: код, который нельзя случайно или намеренно испортить».

8 Слово static

`static` означает «принадлежит классу, а не объекту». Статическое поле существует в одном экземпляре на весь класс, статический метод вызывается через имя класса и не имеет `this`, поэтому не видит обычные поля объекта.

Что	Смысл	Пример в проекте
static-поле	одно на класс, общее для всех объектов	`protected static final ObjectMapper JSON` в `AbstractApiController`
static final	константа	`ALPHA = 0.05`, `TOP_SUBDIMS = 5`
static-метод	функция без объекта	`FinalAnalitikMathematics.mean(x)`, `FinalFormulaParser.parse(...)`
статическая фабрика	static-метод, создающий объект нужного класса	`AbstractSegmentComparisonTest.compare`, `AbstractAnomalyAnalysis.create`
static nested class	класс внутри класса, не привязанный к объекту внешнего	`FinalBranchBuilder.MetricTest`
import static	вызывать static-методы без имени класса	`import static ...FinalFormatRu.fmt` в тестах

Старый код злоупотреблял `static`: всё, от тестов до анализов, было статическими функциями. Теперь `static` остался там, где он уместен: чистая математика без состояния (`mean`, `variance`, `tP`), константы и фабрики. Всё, у чего есть состояние, стало объектами.

ЗАПОМНИ

1. `static` — член класса, а не объекта; нет `this`.
2. Утилитный класс = `final` + `private`-конструктор + static-методы.
3. Статическая фабрика возвращает общий тип и выбирает конкретный класс.

9 Виды классов и структур в Java

В Java «класс» — это целое семейство конструкций. Ниже все основные виды: что это, как выглядит и где встречается у нас.

Вид	Как объявляется	Главное свойство	У нас
обычный класс	`class A`	можно создавать объекты и наследоваться	`ServiceBrainGraph`, исключения
конечный класс	`final class A`	нельзя наследоваться	51 класс `Final…`
абстрактный класс	`abstract class A`	нельзя создать; может иметь абстрактные методы	6 классов `Abstract…`
интерфейс	`interface I`	контракт; класс реализует много интерфейсов	4 `Interface…`
функциональный интерфейс	`@FunctionalInterface interface I`	один абстрактный метод → можно лямбдой	`InterfaceAiTool`
перечисление	`enum E { A, B }`	закрытый набор объектов-констант	4 `Enum…`
вложенный статический	`static class B` внутри A	не привязан к объекту внешнего	`FinalBranchBuilder.MetricTest`
внутренний (inner)	`class B` внутри A	привязан к объекту внешнего класса	—
локальный	класс внутри метода	виден только в методе	—
анонимный	`new I() { ... }`	класс без имени на одно место	заменены лямбдами
обобщённый	`class Box<T>`	параметр типа	используем `List<T>`, `Map<K,V>`
исключение	`class E extends RuntimeException`	можно бросать через `throw`	2 `Exception…`
запись (record)	`record P(double x)`	неизменяемый класс данных, Java 16+	нет, проект на Java 11
запечатанный (sealed)	`sealed interface I permits A, B`	список наследников закрыт, Java 17+	нет, проект на Java 11
аннотация	`@interface Name`	метка для компилятора или фреймворка	используем чужие: `@Service`, `@Override`

9.1. Перечисления (enum)

Enum в Java — полноценный класс, у которого заранее известен и закрыт список объектов. У него могут быть поля, конструктор (всегда закрытый) и методы. Каждая константа может даже переопределить метод по-своему — это называется **тело, зависящее от константы** (constant-specific body). Enum нельзя создать через `new` и нельзя от него наследоваться. Встроенные методы: `values()` — все константы, `valueOf("NAME")` — константа по имени, `name()` и `ordinal()` — имя и порядковый номер. Enum можно использовать в `switch`.

Enum с полями и enum с абстрактным методом · запускается как есть

```java
enum Scale {
    CORRELATION(0.1, 0.3, 0.5),
    DIFFERENCE(0.2, 0.5, 0.8);

    private final double weak;
    private final double medium;
    private final double strong;

    Scale(double weak, double medium, double strong) {
        this.weak = weak;
        this.medium = medium;
        this.strong = strong;
    }

    String level(double effect) {
        double size = Math.abs(effect);
        if (size >= strong) return "сильная";
        if (size >= medium) return "средняя";
        if (size >= weak) return "слабая";
        return "нет";
    }
}

enum Operator {
    PLUS("+") {
        double apply(double a, double b) {
            return a + b;
        }
    },
    TIMES("*") {
        double apply(double a, double b) {
            return a * b;
        }
    };

    final String symbol;

    Operator(String symbol) {
        this.symbol = symbol;
    }

    abstract double apply(double a, double b);
}

public class Main {
    public static void main(String[] args) {
        System.out.println("ρ = 0,42 → " + Scale.CORRELATION.level(0.42));
        System.out.println("g = 0,42 → " + Scale.DIFFERENCE.level(0.42));
        for (Operator op : Operator.values()) {
            System.out.println("6 " + op.symbol + " 7 = " + op.apply(6, 7) + "  (" + op.name() +
", номер " + op.ordinal() + ")");
        }
        Operator op = Operator.valueOf("TIMES");
        switch (op) {
            case PLUS:
                System.out.println("сложение");
                break;
            case TIMES:
                System.out.println("умножение");
                break;
        }
    }
}
```

Вывод программы

```
ρ = 0,42 → средняя
g = 0,42 → слабая
6 + 7 = 13.0  (PLUS, номер 0)
6 * 7 = 42.0  (TIMES, номер 1)
умножение
```

В проекте шкала силы связи стала перечислением `EnumEffectScale`: две константы — для корреляций и для разниц, у каждой свои пороги. Две старые функции `levelCorr` и `levelD` превратились в один метод `level`.

Стало · mathematics/EnumEffectScale.java — файл целиком

```java
package com.analytics.brain.mathematics;

public enum EnumEffectScale {

    CORRELATION(0.1, 0.3, 0.5, 1.0),
    DIFFERENCE(0.2, 0.5, 0.8, 1.2);

    private final double weak;
    private final double medium;
    private final double strong;
    private final double fullStrength;

    EnumEffectScale(double weak, double medium, double strong, double fullStrength) {
        this.weak = weak;
        this.medium = medium;
        this.strong = strong;
        this.fullStrength = fullStrength;
    }

    public String level(double effect) {
        double size = Math.abs(effect);
        if (size >= strong) return "strong";
        if (size >= medium) return "medium";
        if (size >= weak) return "weak";
        return "none";
    }

    public double strength(double effect) {
        return Math.min(1, Math.abs(effect) / fullStrength);
    }
}
```

А `EnumFormulaOperator` — пример тела, зависящего от константы: у каждого оператора свой `apply`. Так 15 веток старого `switch` стали 15 маленькими методами, каждый рядом со своим именем.

Стало · каждая константа со своим методом · formula/EnumFormulaOperator.java, строки 7–26

```
public enum EnumFormulaOperator {

    OR {
        @Override
        public Object apply(Object a, Object b) {
            return bool(a) || bool(b);
        }
    },
    AND {
        @Override
        public Object apply(Object a, Object b) {
            return bool(a) && bool(b);
        }
    },
    GREATER_OR_EQUAL {
        @Override
        public Object apply(Object a, Object b) {
            return bothNumbers(a, b) ? number(a) >= number(b) : sameText(a, b);
        }
    },
```

9.2. Лямбды, функциональные интерфейсы и ссылки на методы

Если у интерфейса ровно один абстрактный метод, вместо целого класса можно написать **лямбду** — короткую функцию: `(ds, args, emit) -> show(...)`. Компилятор сам создаст объект, реализующий интерфейс. Ещё короче — **ссылка на метод**: `this::createReport` означает «реализация — вот этот существующий метод».

Так устроен реестр ИИ-инструментов: словарь `Map<String, InterfaceAiTool>`, где ключ — имя инструмента, а значение — объект-лямбда. Это паттерн **«Стратегия»**: поведение хранится как объект и выбирается по ключу.

Стало · лямбды как объекты · ai/ServiceAiTools.java, строки 53–63

```
private void registerTools() {
    tools.put("pivot_table", (ds, args, emit) ->
            show(FinalQueryEngine.pivot(ds, JSON.treeToValue(args, FinalPivotSpec.class)),
emit));
    tools.put("chart", (ds, args, emit) ->
            show(FinalQueryEngine.chart(ds, JSON.treeToValue(args, FinalChartSpec.class)),
emit));
    tools.put("compare_segments", (ds, args, emit) ->
            show(FinalQueryEngine.compare(ds, args.path("metric").asText(),
args.path("group_by").asText(), conds(args)), emit));
    tools.put("funnel", (ds, args, emit) ->
            show(FinalQueryEngine.funnel(ds, args.path("by").asText(null), conds(args)), emit));
    tools.put("data_quality", (ds, args, emit) ->
            show(new FinalQualityAnalysis(ds).run(), emit));
```

Стало · ссылки на методы · ai/ServiceAiTools.java, строки 83–87

```
tools.put("create_report", this::createReport);
tools.put("create_presentation", this::createPresentation);
tools.put("add_column", this::addColumn);
tools.put("filter_dataset", this::filterDataset);
tools.put("graph_action", (ds, args, emit) -> graphAction(args, emit));
```

9.3. Вложенные классы — четыре вида

Статический вложенный класс — обычный класс, просто объявленный внутри другого для порядка; с внешним объектом не связан. **Внутренний** (без `static`) — каждый его объект привязан к объекту внешнего

класса и видит его поля; создаётся странной записью `outer.new Counter()`. **Локальный** — объявлен прямо в методе. **Анонимный** — класс без имени, созданный в одном выражении; сегодня почти всегда заменяется лямбдой.

Все четыре вида вложенных классов, лямбда и ссылка на метод · запускается как есть

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.function.Function;

public class Main {
    static final class Stat {
        final String name;
        final double value;

        Stat(String name, double value) {
            this.name = name;
            this.value = value;
        }
    }

    class Counter {
        int calls;

        void hit() {
            calls++;
            total++;
        }
    }

    int total;

    public static void main(String[] args) {
        Stat nested = new Stat("static nested", 1);

        Main outer = new Main();
        Main.Counter inner = outer.new Counter();
        inner.hit();
        inner.hit();

        class Pair {
            final String left;
            final String right;

            Pair(String left, String right) {
                this.left = left;
                this.right = right;
            }
        }
        Pair local = new Pair("local", "class");

        Comparator<Stat> anonymous = new Comparator<Stat>() {
            @Override
            public int compare(Stat a, Stat b) {
                return Double.compare(a.value, b.value);
            }
        };
        Comparator<Stat> lambda = (a, b) -> Double.compare(a.value, b.value);
        Function<Stat, String> reference = Main::label;

        List<Stat> list = new ArrayList<>(List.of(new Stat("B", 2), new Stat("A", 1)));
        list.sort(anonymous);
        list.sort(lambda.reversed());

        System.out.println(nested.name + " | inner: " + inner.calls + " / outer: " + outer.total
                + " | " + local.left + " " + local.right + " | " +
reference.apply(list.get(0)));
    }

    static String label(Stat s) {
        return s.name + "=" + s.value;
```

```
    }
}
```

Вывод программы

```
static nested | inner: 2 / outer: 2 | local class | B=2.0
```

Стало · статический вложенный класс в проекте · service/FinalBranchBuilder.java, строки 42–55

```
private static final class MetricTest {

    final FinalColumn column;
    final AbstractSegmentComparisonTest test;
    final double[] segment;
    final double[] others;

    MetricTest(FinalColumn column, AbstractSegmentComparisonTest test, double[] segment,
double[] others) {
        this.column = column;
        this.test = test;
        this.segment = segment;
        this.others = others;
    }
}
```

9.4. Исключения — тоже классы

Ошибка в Java — это объект класса-наследника `Throwable`. Ветка `Error` — сбои виртуальной машины, их не ловят. Ветка `Exception` делится на **проверяемые** (checked: компилятор требует либо поймать, либо объявить `throws`, например `IOException`) и **непроверяемые** — наследники `RuntimeException`, объявлять их не обязательно.

Наши исключения наследуют непроверяемые классы JDK: `ExceptionAnalitikData extends IllegalArgumentException` и `ExceptionDatasetNotFound extends NoSuchElementException`. Благодаря наследованию их ловят те же обработчики, что и родительские типы: `@ExceptionHandler(NoSuchElementException.class)` в `AbstractApiController` поймает и наш `ExceptionDatasetNotFound` — это полиморфизм в обработке ошибок.

Конструкция **try-with-resources** `try (Resource r = ...)` автоматически вызывает `close()` у ресурсов в обратном порядке, даже при ошибке. Это ближайший аналог деструктора C++ в Java.

Своё исключение, try-with-resources, catch и finally · запускается как есть

```java
class DatasetNotFound extends RuntimeException {
    DatasetNotFound(String id) {
        super("Набор данных не найден: " + id);
    }
}

class Resource implements AutoCloseable {
    private final String name;

    Resource(String name) {
        this.name = name;
        System.out.println("открыт " + name);
    }

    @Override
    public void close() {
        System.out.println("закрыт " + name);
    }
}

public class Main {
    static String find(String id) {
        if (!id.equals("demo")) throw new DatasetNotFound(id);
        return "демо-набор";
    }

    public static void main(String[] args) {
        try (Resource a = new Resource("файл"); Resource b = new Resource("соединение")) {
            System.out.println(find("demo"));
            System.out.println(find("x42"));
        } catch (DatasetNotFound e) {
            System.out.println("поймали: " + e.getMessage());
        } finally {
            System.out.println("finally выполняется всегда");
        }
    }
}
```

Вывод программы

```
открыт файл
открыт соединение
демо-набор
закрыт соединение
закрыт файл
поймали: Набор данных не найден: x42
finally выполняется всегда
```

Стало · бросаем своё исключение, ловим по родительскому типу · api/AbstractApiController.java, строки 23–36

```java
protected final FinalDataset dataset(String id) {
    FinalDataset d = brain.store.get(id);
    if (d == null) throw new ExceptionDatasetNotFound("Набор данных не найден: " + id +
notFoundHint());
    return d;
}

protected abstract String notFoundHint();

protected abstract String unknownErrorText(Exception e);

@ExceptionHandler(NoSuchElementException.class)
public ResponseEntity<Map<String, Object>> notFound(Exception e) {
    return ResponseEntity.status(404).body(Map.of("error", e.getMessage()));
}
```

9.5. Record и sealed — новые виды (не в нашем проекте)

Проект собран под Java 11, поэтому двух современных конструкций в нём нет, но спросить о них могут. **Record** (Java 16) — неизменяемый класс данных в одну строку: компилятор сам создаёт поля `private final`, конструктор, методы доступа, `equals`, `hashCode` и `toString`. **Sealed** (Java 17) — класс или интерфейс, который сам перечисляет своих наследников в `permits`. Вместе с сопоставлением по образцу (`instanceof Num num`) они позволяют описать дерево формулы очень коротко:

Record + sealed (компилируется Java 17+) · запускается как есть

```java
sealed interface Node permits Num, Add {
}

record Num(double value) implements Node {
}

record Add(Node left, Node right) implements Node {
}

public class Main {
    static double eval(Node n) {
        if (n instanceof Num num) return num.value();
        Add add = (Add) n;
        return eval(add.left()) + eval(add.right());
    }

    public static void main(String[] args) {
        Node tree = new Add(new Num(2), new Add(new Num(3), new Num(4)));
        System.out.println(tree + " = " + eval(tree));
    }
}
```

Вывод программы

```
Add[left=Num[value=2.0], right=Add[left=Num[value=3.0], right=Num[value=4.0]]] = 9.0
```

9.6. Аннотации и Spring

Аннотация — метка, которую читает компилятор или фреймворк. `@Override` и `@FunctionalInterface` проверяет компилятор. `@Service` и `@RestController` читает Spring: он сам создаёт по одному объекту каждого такого класса (шаблон «одиночка», singleton) и передаёт их друг другу через конструктор. Это называется **внедрением зависимостей** (dependency injection): класс не создаёт своих помощников через `new`, а получает готовых.

Стало · Spring передаёт три сервиса в конструктор · ai/ServiceAiTools.java, строки 39–44

```java
public ServiceAiTools(ServiceBrainGraph brain, ServiceAiClient client, ServiceAiDocuments
documents) {
    this.brain = brain;
    this.client = client;
    this.documents = documents;
    registerTools();
}
```

9.7. Структуры данных, которые мы используем

Структура	Что это	Где у нас
`double[]`, `String[]`	массив фиксированной длины, самый быстрый доступ	значения колонок в `FinalColumn`
`ArrayList`	список-массив, растёт автоматически	узлы и связи графа, строки таблиц

`LinkedHashMap`	словарь, помнящий порядок вставки	результаты анализов, реестр инструментов
`ConcurrentHashMap`	словарь, безопасный для нескольких потоков	хранилище наборов `store` в сервисе
`LinkedHashSet`	множество без повторов с порядком	заголовки колонок JSON
`List.of(...)`, `Map.of(...)`	неизменяемые коллекции	ответы об ошибках

Обрати внимание: поля и параметры объявлены через **интерфейсы** `List` и `Map`, а конкретный класс указан только при создании: `Map<String, InterfaceAiTool> tools = new LinkedHashMap<>()`. Это «программирование на уровне интерфейсов» — тот же полиморфизм: реализацию можно заменить, не трогая остальной код.

9.8. Паттерны проектирования в проекте

Паттерн	Суть одной фразой	Где
Шаблонный метод	родитель задаёт порядок шагов (final), наследник заполняет шаг (abstract)	`AbstractAnalitikAnalysis.run / compute`
Фабричный метод	статический метод возвращает общий тип и выбирает класс	`compare`, `create`, `parse`
Интерпретатор	выражение — дерево объектов, каждый узел вычисляет себя	пакет `formula`
Стратегия	поведение хранится как объект и подменяется	словарь `InterfaceAiTool`, `InterfaceDataParser`
Внедрение зависимостей	объект получает помощников через конструктор	все `Service…` и `Controller…`
Утилитный класс	final + private-конструктор + static-методы	`FinalAnalitikMathematics`, `FinalFormatRu`
DTO	класс только для передачи данных	пакет `graph`, `FinalChatRequest`

ЗАПОМНИ

1. Виды: class, final, abstract, interface, enum, вложенные (4 вида), generics, исключения, record и sealed (новые).
2. Enum — класс с закрытым списком объектов; может иметь поля, конструктор и свой метод у каждой константы.
3. Лямбда реализует функциональный интерфейс; у нас так хранятся 17 ИИ-инструментов.
4. Исключения — классы; checked надо объявлять, unchecked (RuntimeException) — нет.

10 Классы в C++ и в Java

Идеи ООП одинаковые, а устройство разное. Главное различие: в C++ программист сам управляет памятью и объект может лежать прямо в переменной; в Java объект всегда в куче, переменная — ссылка, а память освобождает сборщик мусора.

10.1. Объект, конструктор и деструктор

В C++ объект можно создать **на стеке**: `Column onStack("Выручка");`. Он живёт до закрывающей фигурной скобки своего блока, а потом автоматически вызывается **деструктор** `~Column()`. Объект **в куче** создаётся `new` и живёт, пока программист не вызовет `delete`; забыл — утечка памяти, вызвал дважды — падение. Поэтому в современном C++ вместо «голых» указателей используют умные: `std::unique_ptr` сам вызывает `delete` в своём деструкторе.

Присваивание `Column copy = onStack;` в C++ **копирует весь объект** (работает конструктор копирования), и изменение копии не трогает оригинал. В Java та же запись копирует только ссылку (сравни с примером из раздела 2). Обрати внимание на вывод: для копии конструктор не напечатал сообщение — сработал автоматически созданный конструктор копирования, — а деструктор «Выручка» вызвался дважды: для копии и для оригинала.

C++: стек, куча, деструктор, копирование · запускается как есть

```cpp
#include <iostream>
#include <string>
#include <vector>

class Column {
public:
    Column(std::string name) : name_(std::move(name)) {
        std::cout << "конструктор " << name_ << "\n";
    }
    ~Column() {
        std::cout << "деструктор " << name_ << "\n";
    }
    void add(double v) { values_.push_back(v); }
    double mean() const {
        double s = 0;
        for (double v : values_) s += v;
        return values_.empty() ? 0 : s / values_.size();
    }

private:
    std::string name_;
    std::vector<double> values_;
};

int main() {
    Column onStack("Выручка");
    onStack.add(100);
    onStack.add(300);
    std::cout << "среднее " << onStack.mean() << "\n";

    Column* onHeap = new Column("NPS");
    delete onHeap;

    {
        Column copy = onStack;
        copy.add(1000);
        std::cout << "копия " << copy.mean() << ", оригинал " << onStack.mean() << "\n";
    }
    std::cout << "конец main\n";
}
```

Вывод программы

```
конструктор Выручка
среднее 200
конструктор NPS
деструктор NPS
копия 466.667, оригинал 200
деструктор Выручка
конец main
деструктор Выручка
```

Приём «ресурс захватывается в конструкторе и освобождается в деструкторе» называется **RAII** и является основой C++. В Java деструкторов нет; для файлов и соединений используют `try-with-resources` и метод `close()` (раздел 9.4).

10.2. Наследование и полиморфизм

Синтаксис наследования в C++: `class WelchTest final : public AbstractSegmentTest`. Слово `public` здесь — **вид наследования**: открытые члены родителя остаются открытыми. Бывает ещё `protected` и `private`-наследование, в Java такого нет. Конструктор родителя вызывается в **списке инициализации** после двоеточия: `: AbstractSegmentTest(p)` — аналог `super(p)`.

Самое важное отличие: в C++ метод полиморфен, **только если он объявлен `virtual`**. Без этого слова вызов выбирается по типу переменной ещё при компиляции. И работает полиморфизм только через указатель или ссылку — если скопировать наследника в переменную родительского типа, лишняя часть «срежется» (object slicing). В Java все методы виртуальны по умолчанию, а объекты никогда не копируются при присваивании, поэтому обеих ловушек нет.

C++: метод без virtual, метод с virtual и срезка · запускается как есть

```cpp
#include <iostream>
#include <string>

class Base {
public:
    std::string plain() const { return "Base::plain"; }
    virtual std::string virt() const { return "Base::virt"; }
    virtual ~Base() = default;
};

class Derived : public Base {
public:
    std::string plain() const { return "Derived::plain"; }
    std::string virt() const override { return "Derived::virt"; }
};

int main() {
    Derived d;
    Base& ref = d;
    std::cout << ref.plain() << " | " << ref.virt() << "\n";
    Base sliced = d;
    std::cout << sliced.virt() << "  (срезка: скопирована только часть Base)\n";
}
```

Вывод программы

```
Base::plain | Derived::virt
Base::virt  (срезка: скопирована только часть Base)
```

Если у класса есть виртуальные функции, его деструктор тоже должен быть `virtual`. Иначе при удалении наследника через указатель на родителя вызовется только деструктор родителя, и часть объекта не освободится. Ниже — тот же пример с тестами, что в разделе 6, но на C++. Сравни порядок: при создании сначала база, потом наследник; при удалении — наоборот.

C++: абстрактный класс, virtual, override, final, фабрика · запускается как есть

```cpp
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class AbstractSegmentTest {
public:
    explicit AbstractSegmentTest(double p) : p_(p) {
        std::cout << "  база создана\n";
    }
    virtual ~AbstractSegmentTest() {
        std::cout << "  база удалена\n";
    }
    virtual std::string symbol() const = 0;
    bool significant() const { return p_ < 0.05; }
    virtual std::string describe() const {
        return symbol() + (significant() ? " значимо" : " не значимо");
    }

protected:
    double p_;
};

class WelchTest final : public AbstractSegmentTest {
public:
    explicit WelchTest(double p) : AbstractSegmentTest(p) {
        std::cout << "  WelchTest создан\n";
    }
    ~WelchTest() override {
        std::cout << "  WelchTest удалён\n";
    }
    std::string symbol() const override { return "t"; }
};

class TwoProportionTest final : public AbstractSegmentTest {
public:
    using AbstractSegmentTest::AbstractSegmentTest;
    std::string symbol() const override { return "z"; }
    std::string describe() const override {
        return "доли: " + AbstractSegmentTest::describe();
    }
};

std::unique_ptr<AbstractSegmentTest> compare(bool countLike, double p) {
    if (countLike) return std::make_unique<TwoProportionTest>(p);
    return std::make_unique<WelchTest>(p);
}

int main() {
    std::vector<std::unique_ptr<AbstractSegmentTest>> tests;
    tests.push_back(compare(false, 0.01));
    tests.push_back(compare(true, 0.2));
    for (const auto& t : tests) std::cout << t->describe() << "\n";
    std::cout << "выход из main:\n";
}
```

Вывод программы

```
  база создана
  WelchTest создан
  база создана
t значимо
доли: z не значимо
выход из main:
  WelchTest удалён
  база удалена
  база удалена
```

10.3. Абстрактные классы и «интерфейсы» в C++

Ключевых слов `abstract` и `interface` в C++ нет. Класс становится абстрактным, если в нём есть хотя бы одна **чисто виртуальная функция**: `virtual std::string symbol() const = 0;`. Роль интерфейса играет класс, в котором все функции чисто виртуальные. Зато в C++ разрешено **множественное наследование** классов, поэтому класс может унаследовать несколько таких «интерфейсов» — это аналог `implements A, B` в Java. Проблему ромба в C++ решают виртуальным наследованием (`class B : virtual public A`).

C++: два «интерфейса» и множественное наследование · запускается как есть

```cpp
#include <iostream>
#include <string>

class Named {
public:
    virtual std::string name() const = 0;
    virtual ~Named() = default;
};

class Significant {
public:
    virtual double pValue() const = 0;
    virtual ~Significant() = default;
};

class SpearmanTest : public Named, public Significant {
public:
    std::string name() const override { return "Спирмен"; }
    double pValue() const override { return 0.003; }
};

int main() {
    SpearmanTest s;
    const Named& n = s;
    const Significant& sig = s;
    std::cout << n.name() << ", p = " << sig.pValue() << "\n";
}
```

Вывод программы

```
Спирмен, p = 0.003
```

10.4. final, override и const

В C++11 появились те же слова, что в Java, но пишутся они *после* имени: `class WelchTest final`, `std::string symbol() const override`, `virtual bool significant() const final`. Компилятор ругается похоже:

```
g++: cannot derive from 'final' base 'WelchTest' in derived type 'Hacked'
```

```
g++: virtual function 'virtual bool Hacked::significant() const' overriding final function
```

```
g++: cannot declare variable 't' to be of abstract type 'AbstractTest'
```

Роль `final`-поля в C++ играет `const`. Но у `const` есть ещё смысл, которого нет в Java: `const` после метода (`double mean() const`) обещает, что метод не меняет объект, а `const`-ссылка запрещает менять объект через неё.

10.5. Большая таблица различий

Тема	C++	Java
Где живут объекты	на стеке или в куче — решает программист	всегда в куче; в переменной — ссылка
Освобождение памяти	деструктор, `delete`, умные указатели, RAII	сборщик мусора; деструкторов нет
Присваивание объекта	копирует объект (конструктор копирования)	копирует ссылку
Правило трёх / пяти	копирование, перемещение, деструктор — продумать	не нужно
Виртуальность методов	только с `virtual`	все методы экземпляра по умолчанию
Полиморфизм	через указатель или ссылку; иначе срезка	всегда, срезки нет
Абстрактный класс	есть функция `= 0`	слово `abstract`
Интерфейс	класс из чисто виртуальных функций	слово `interface`
Множественное наследование	разрешено для классов	только интерфейсов
Вид наследования	`public` / `protected` / `private`	один вид
Доступ	секции `public:`, `private:`; по умолчанию `private`	слово перед каждым членом; по умолчанию package-private
`struct`	то же, что класс, но доступ по умолчанию `public`	нет; близко — `record`
Вызов родителя	список инициализации `: Base(x)`, `Base::f()`	`super(x)`, `super.f()`
Запрет наследования / переопределения	`final` после имени (C++11)	`final` перед объявлением
Проверка переопределения	`override` после сигнатуры	`@Override` перед методом
Неизменяемость	`const` (переменные, методы, ссылки)	`final` (только переменные и поля)
`friend`	есть — доступ к закрытым членам	нет
Перегрузка операторов	есть (`operator+`)	нет
Шаблоны / дженерики	шаблоны: код создаётся для каждого типа	дженерики: стирание типов, один код
Файлы	объявление в .h, реализация в .cpp	один класс верхнего уровня — один .java
Общий корень	нет	`java.lang.Object`
Исключения	можно бросить что угодно, даже `int`	только наследники `Throwable`; есть checked

ЗАПОМНИ

1. C++: объект может быть значением на стеке; есть деструктор; память — забота программиста (RAII, умные указатели).
2. C++: полиморфизм только для `virtual` и только через указатель/ссылку; иначе статическое связывание и срезка.
3. C++: абстрактный класс = есть `= 0`; интерфейсов нет, зато множественное наследование.
4. Java: всё через ссылки, все методы виртуальны, GC, один родитель + много интерфейсов.

ЕСЛИ СПРОСЯТ — ОТВЕТ ЗА 20 СЕКУНД

«Главные отличия: в C++ объекты могут жить на стеке и удаляются деструктором, в Java — только в куче и их удаляет сборщик мусора. В C++ метод полиморфен, только если он virtual, в Java — всегда. В C++ есть множественное наследование и нет интерфейсов, в Java наоборот. final и override в C++11 пишутся после имени».

11 Плюсы и цена объектного подхода

11.1. Что дала перестройка проекту

Плюс	Как это видно в коде
Расширяемость	новый тест = новый класс-наследник; менять `applyFdr`, `FinalBranchBuilder` и графы не нужно
Нет дублирования	правило значимости и классификация силы написаны один раз в родителе; шкала — в enum
Защита правил	`final` не даёт подменить формулы, правило значимости и порядок шагов анализа
Читаемость	файл = одно понятие; тип класса виден по имени; самый длинный класс — 848 строк вместо 1 131
Меньше ошибок	ни одного `instanceof` с приведением типа; ошибку в переопределении ловит `@Override`
Скорость формул	текст формулы разбирается один раз, а не для каждой строки
Тестируемость	маленький класс с одной задачей легко проверить отдельно
Понятные ошибки	свои исключения: 400 — ошибка в данных, 404 — нет набора

11.2. Честно о минусах

Классов стало больше — 74 вместо 18, и чтобы понять путь запроса, нужно пройти через несколько файлов. Имена длинные. Приставки `Final`, `Abstract`, `Interface` — не стандарт Java, а учебное соглашение проекта. Публичные поля в DTO — компромисс ради Jackson. Глубокие иерархии вообще опасны: изменение родителя затрагивает всех наследников («проблема хрупкого базового класса»), поэтому у нас максимум три уровня, а там, где отношения «является» нет, использована композиция.

> **Четыре принципа ООП одной строкой**
>
> **Инкапсуляция** — прячем устройство, показываем минимум (модификаторы доступа). **Наследование** — берём готовое и расширяем (`extends`, `super`). **Абстракция** — описываем главное, детали оставляем наследникам (`abstract`, `interface`). **Полиморфизм** — один вызов, разное поведение (override, динамическое связывание).

Шпаргалка на одну страницу

Класс / объект	чертёж / экземпляр; `new` + конструктор; переменная = ссылка; GC
`this` / `super`	текущий объект / родитель; `this(...)`, `super(...)` — первая строка
Инкапсуляция	`private` → пакет → `protected` → `public`
Наследование	`extends`, один родитель, корень `Object`; конструкторы не наследуются; сборка сверху вниз
`@Override`	та же сигнатура; доступ не сужать; компилятор проверяет
Абстрактный класс	нельзя `new`; поля, конструктор, обычные + abstract-методы; «является»
Интерфейс	контракт; `implements` много; default/static-методы; «умеет»
Полиморфизм	тип переменной — что можно вызвать; класс объекта — чья версия выполнится
Override / overload	динамический (при выполнении) / статический (при компиляции)
`final` класс	нельзя наследоваться: `FinalWelchTest`, `String`
`final` метод	нельзя переопределить: `significant()`, `run()`
`final` поле	одно присваивание; `static final` = константа; ссылка ≠ неизменяемый объект
`static`	член класса, без `this`; утилиты, константы, фабрики
enum	закрытый список объектов; поля, конструктор, метод у каждой константы
Лямбда	реализация функционального интерфейса (1 метод); `this::method`
Вложенные	static nested, inner, local, anonymous
Исключения	`Throwable` → `Exception` (checked) → `RuntimeException` (unchecked)
Паттерны у нас	шаблонный метод, фабрика, интерпретатор, стратегия, DI
C++ vs Java	стек+деструктор / куча+GC; `virtual` явно / всегда; мн. наследование / интерфейсы
C++ абстракция	`virtual f() = 0`; интерфейс = класс из таких функций
C++ final/override	после имени: `class A final`, `f() override`; `const` ≈ final

НАШ ПРОЕКТ

Было	18 классов-«папок», static-функции, instanceof, switch на 17 веток, формула разбиралась на каждой строке
Стало	74 класса, 11 пакетов: 51 Final, 6 Abstract, 4 Interface, 4 Enum, 5 Service, 2 Controller, 2 Exception
Иерархия тестов	`InterfaceSignificanceTest` ⇠ `AbstractAnalitikMathematicsTest` ← Correlation / SegmentComparison ← 6 Final
Шаблонный метод	`AbstractAnalitikAnalysis`: `final run()` + `abstract compute()`
Полиморфизм	`compare` → Welch / TwoProportion; `parseAuto` → Csv / Json; `applyFdr`; дерево формулы
Проверка	723 запроса и 80 формул — ответы старой и новой версии совпали